CAO: Prediction-Aware Task Scheduling for Heterogeneous Computing

Project explanation and presentation guide (study material, not a research paper)

Source version	Git branch **revival/resource-state-aware**, commit **9d8ea22** (committed baseline). The audit session's uncommitted edits were inspected read-only and are labelled **AUDIT WIP**.
Guide written	2026-11-01. Nothing in the project was modified.
Result policy	Only numbers found in the project's own records are used (mainly PROJECT_BACKBONE.md, docs/revival/*, data/results/*). Anything not finished is labelled **PENDING**.
Status labels	**VERIFIED** = measured, raw logs stored. **ESTIMATE** = simulation/bound. **AUDIT WIP** = uncommitted, not final. **PENDING** = final experiments not finished.

One-sentence summary to memorise: CAO is a Python framework that runs a workflow (a DAG of tasks) on a laptop's fast CPU cores, slow CPU cores, a GPU and an emulated remote worker, and tests whether a scheduler that *predicts* each task's runtime with a small ML model finishes the whole workflow sooner than simple rules. Honest result: it clearly beats static and naive schedulers (17% to 45% faster) and beats the simple greedy baseline B4 only when task cost is hidden inside task parameters; it never beat the best hand-written rule, and the GPU/live-state ideas were not shown to help.

Contents

- 1 Introduction and objectives
- 2 Fundamentals and terminology
- 3 System architecture
- 4 End-to-end workflow
- 5 Methodology
- 6 DAG, workloads, dependencies
- 7 ML runtime estimator and metrics
- 8 Scheduling algorithms
- 9 CPU, GPU, CUDA, remote worker, offloading
- 10 Directory and important files
- 11 Code walkthrough (stage -> class/function)
- 12 Frontend and demonstration
- 13 Experiments, metrics and verified results
- 14 Base papers and related work
- 15 Limitations, future work, faculty Q&A

1. Introduction, problem and objectives

1.1 Real-world analogy

Imagine a kitchen with **2 fast chefs**, **7 slow chefs**, 1 specialist machine (the GPU) and a helper in another building (the remote worker). Dinner is a recipe with dependent steps: you cannot plate before cooking. A **scheduler** is the head chef who decides, every time a step becomes possible, *which chef* does it and *in what order*. A lazy head chef gives each step to the first free chef. A smarter head chef *guesses how long each step will take on each chef* and gives the long, critical steps first to the right chef. CAO builds the kitchen, the recipes and several head chefs, then measures who finishes dinner first.

1.2 Problem and motivation

- Modern machines are **heterogeneous**: different cores and accelerators have different speeds, and moving data between them costs time.
- Static schedulers decide everything before the run using fixed rules; they ignore what actually happens at runtime.
- Goal: a scheduler that is **prediction-aware** and **dynamic** (decides during the run) and lowers **end-to-end makespan** (total time from start to finish of the workflow). Better prediction numbers alone are not the goal.

1.3 Objectives

- Build a real runtime: DAG, coordinator, worker pool with P-core, E-core, CUDA and remote workers, event logs.
- Build runtime estimators (constant table, analytic, history, ridge/ML, oracle) and many scheduling policies (B0 to B7, EFT, HEFT, PEFT, rank-based).
- Evaluate honestly: pre-registered protocols, paired comparisons, bootstrap confidence intervals, Holm correction, keep negative results.

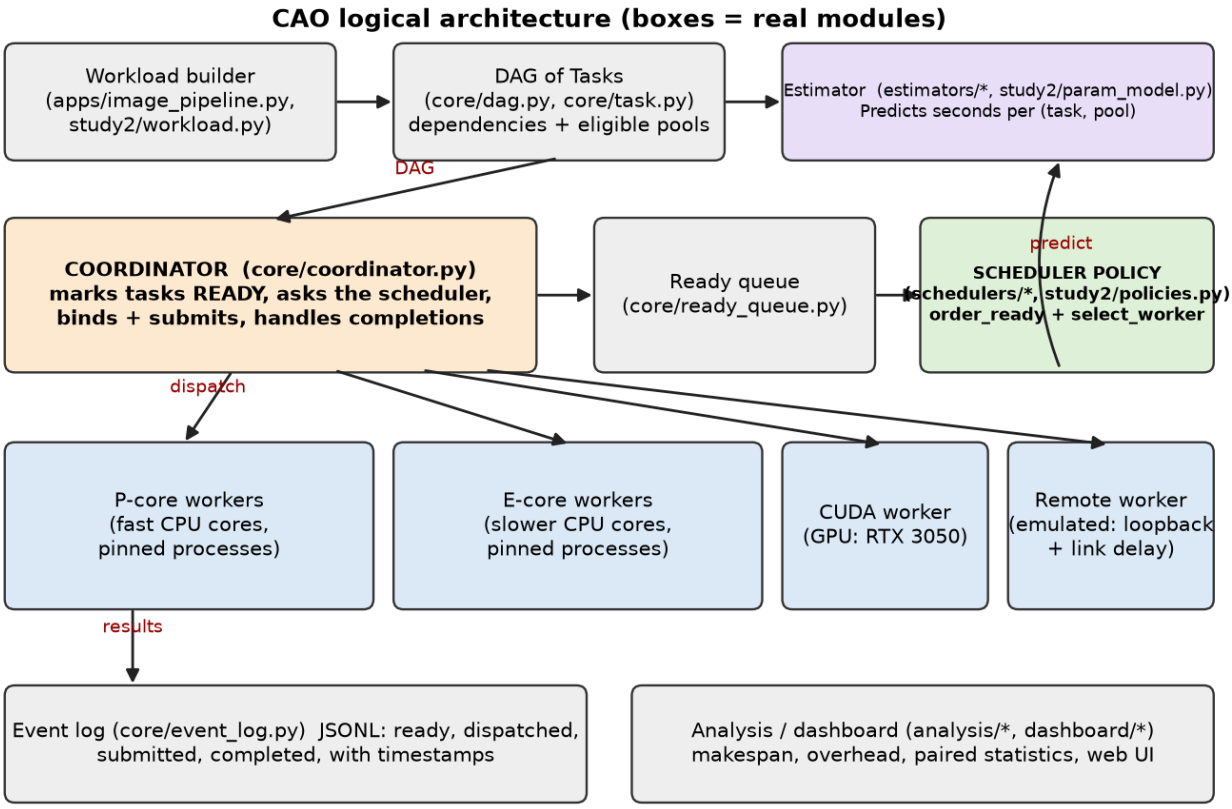
1.4 Expected vs actual contribution

Intended	What the evidence actually supports
Prediction-aware heterogeneous offloading beats static scheduling	**VERIFIED**: learned-prediction policy is 17% to 45% faster than static P-only / table-predictor policies on several pre-registered workloads.
...and beats simple dynamic greedy (B4)	Only where cost is hidden in task parameters (about 20-21% faster). Level with B4 on uniform and two-factor workloads. Montage: valid **null** result.
GPU offloading speeds up workflows	**Not established**. Single GPU tasks can be faster, but workflow-level headroom was ~0% in simulation; only 6.4% of dispatches had a real choice.
Remote/Raspberry Pi offloading	Only **emulated** (loopback + delay shim). No real remote hardware tested.

2. Fundamentals and terminology

Term	Simple meaning in this project
CPU / core	The processor that runs general programs; a core executes one instruction stream at a time.
P-core / E-core	Performance cores are fast; Efficiency cores are slower but numerous. Here: 6 P workers and 7 E workers (coordinator uses one reserved core). Measured P/E speed ratio is about 2.3x to 3.6x for tasks.
GPU / CUDA / kernel	A GPU has thousands of small cores, good for big data-parallel work. CUDA is NVIDIA's programming platform. A **kernel** is the function that runs on the GPU. Here: RTX 3050 6 GB laptop GPU via PyTorch.
Heterogeneous computing	Using different kinds of processors in one system (P, E, GPU, remote).
Task / process / thread	A **task** is one unit of work (one pipeline stage on one image). A **process** is a running program with its own memory; CAO workers are separate pinned processes. A thread is a lighter flow inside a process.
Workflow, DAG, node, edge	A workflow is many tasks with ordering. A **DAG** (directed acyclic graph) draws tasks as **nodes** and dependencies as **edges** (arrows) with no cycles.
Dependency / ready task	Task B depends on A if B needs A's output. A task is **ready** when all its dependencies are done.
Execution time / runtime prediction	How long a task runs on a given worker. The estimator predicts it before the run (features -> seconds).
Scheduling / placement / offloading	Scheduling = choosing order and worker. Placement = choosing the worker. Offloading = sending work to another device (GPU or remote).
Static vs dynamic	Static: whole plan fixed before running (HEFT, PEFT, B0 P-only). Dynamic: decisions made as tasks become ready (B4, PRED_RANK_GREEDY).
Early vs late binding	Early: task is committed to a (possibly busy) worker's queue. Late: task is bound only to an *idle* worker. Early binding cost about 1.7% in the Montage study; late binding is work-conserving (no worker idles while work waits).
Load balancing	Spreading work so no worker idles while another is overloaded.
Data transfer / communication / scheduling overhead	Time to move data (host to GPU, or over a network link); time for messages between coordinator and workers (IPC); time the scheduler spends deciding. Scheduling cost is guarded to stay below 25% of makespan.
Makespan	Time from workflow start to the last task finishing. The main metric. Lower is better.
Critical path	The longest chain of dependent tasks; it sets a lower bound on makespan.
Resource utilisation	Fraction of time workers are busy. Uniform batch workload ran at about 95%.
Remote worker / emulation	A worker in another process behind a delay shim (latency + per-message + bytes/bandwidth) that imitates a network. It is **not** a real second machine.

3. Complete system architecture



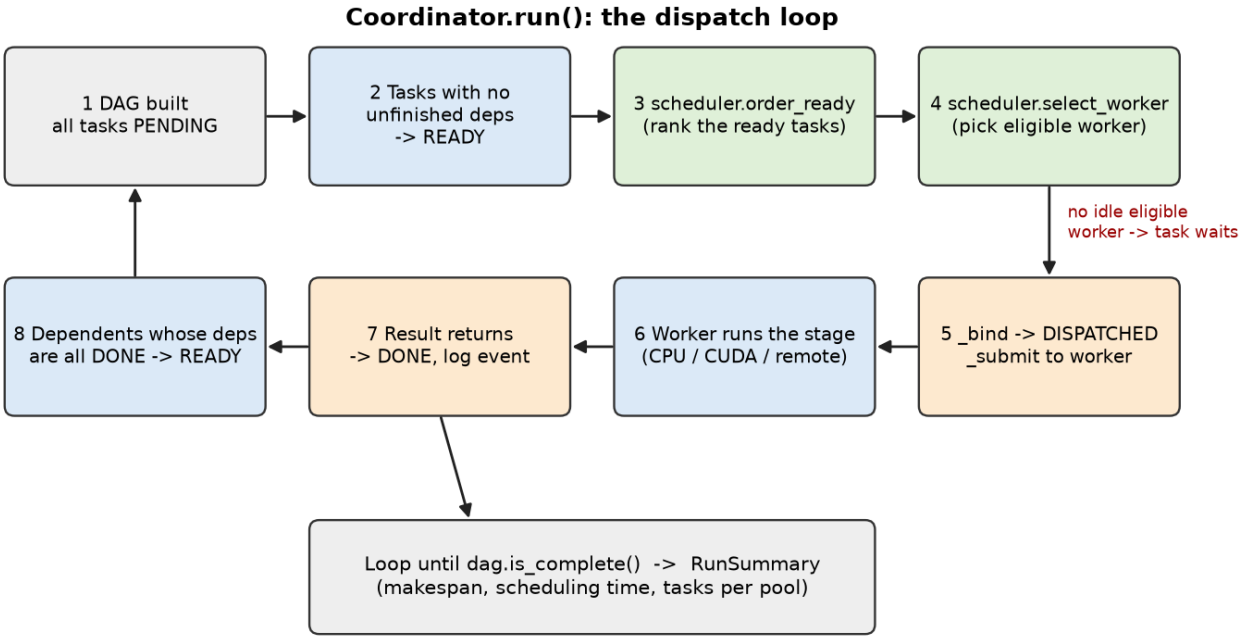
Reading the diagram. The **workload builder** creates a DAG of Task objects. The **coordinator** is the single controller: it keeps tasks in states and a **ready queue**, and on every pass it asks the **scheduler policy** two questions: (1) order_ready - in what order should ready tasks be considered? (2) select_worker - which worker gets this task? The policy may consult an **estimator** for predicted seconds. The coordinator then binds and submits the task to a worker. Workers run the stage and write results to artifact files; every step is written to a JSONL **event log**, which the analysis scripts and dashboard read.

Logical vs real process architecture. Logically there are 'pools' P, E, CUDA, R. In reality each worker is a separate OS process pinned to a CPU core (core/worker_pool.py _worker_loop); the coordinator runs in the main process pinned to a reserved core; the remote worker is a separate process (remote/node.py) reached through a socket proxy (remote/proxy.py) with an artificial link delay.

Task lifecycle (core/task.py: TaskState)

```
PENDING --(all dependencies DONE)--> READY --(scheduler binds)--> DISPATCHED --(worker returns)--> DONE
                                                                          \--> FAILED
Illegal transitions raise InvalidTransition (the coordinator refuses to corrupt the timeline).
```

4. End-to-end workflow, step by step



Numbered walkthrough of one run (Coordinator.run in core/coordinator.py):

- **1. Build.** A workload builder returns a DAG; every task starts PENDING with its dependencies and eligible_pools (which pools may run it).
- **2. Check.** resolve_eligibility verifies every task can run somewhere; the run refuses to start if workers are dead or busy.
- **3. Start.** scheduler.on_run_start runs once (HEFT/PEFT compute their static plan here). Tasks with no dependencies become READY.
- **4. Dispatch pass.** _dispatch_ready_tasks: ask order_ready, then for each task select_worker. If it returns None the task waits.
- **5. Bind + submit.** _bind marks the task DISPATCHED and logs the decision; _submit builds a TaskMessage (CPU variant for P/E, device=cuda for the CUDA worker) and sends it to the worker.
- **6. Execute.** The worker reads input artifacts, runs the stage kernel, writes the output artifact, returns a TaskResult.
- **7. Complete.** _handle_completion marks DONE, logs timings, tells the policy (on_task_completed), and makes newly unblocked dependents READY.
- **8. Finish.** Loop ends when dag.is_complete(); a RunSummary (makespan, scheduling time, tasks per pool) is returned and logged.

What happens in special situations

Situation	What CAO does
Task has unfinished dependencies	Stays PENDING; not in the ready queue; no scheduler call.
Many tasks are ready	order_ready ranks them (B4: FIFO; PRED_RANK_GREEDY: longest predicted remaining chain first).
Worker is busy	Late-binding policies only choose idle workers, so the task waits in the ready queue. Early-binding EFT may queue it behind a busy worker.
Task eligible for CPU only	e.g. img_source: only P and E workers are candidates.

Situation	What CAO does
Task can run on CPU or GPU	Policy compares idle eligible workers; PRED picks the smallest *predicted* time; B4 just takes the first idle by pool priority.
GPU transfer overhead is large	Small inputs make GPU slower than a P core (e.g. edges 256 px: P 6.2 ms vs GPU 7.2 ms). The estimator must include copy/IO cost or the policy mis-places.
Task finishes	State DONE; dependents checked; if all their dependencies are DONE they become READY.
Whole DAG completes	run_finished event with the summary; artifacts are verified against a single-process reference output.

Tiny worked example

DAG A->B, A->C, B->D, C->D (see Section 8.3). t=0: only A is ready. A finishes: B and C become ready at the same time. Two idle workers: both are dispatched. D waits (PENDING) until both B and C are DONE, then it becomes READY and is dispatched. If only one worker is idle, ordering decides: the policy sends the longer-chain task (B) first.

5. Proposed methodology

Step	What happens	Component
1 Workflow/DAG generation	Image-analysis pipeline and synthetic chain workloads	apps/image_pipeline.py, study2/workload.py, core/dag.py
2 Tasks + dependencies	Task objects with params, size, dependencies, eligible pools	core/task.py, core/eligibility.py
3 Profiling / training data	A 'coverage session' runs every stage on every pool and logs kernel seconds	scripts/study2_*.py, event logs in data/events
4 Runtime estimation	Per-(stage,pool) ridge regression on task parameters (ParamModelEstimator)	study2/param_model.py
5 Ready-task identification	Coordinator marks tasks READY when dependencies are DONE	Coordinator._mark_ready
6 Scheduling policy	order_ready ranks tasks (upward rank from predicted times)	study2/policies.py PredictedRankGreedy
7 Worker selection	Idle eligible worker with the smallest predicted time	PredictedRankGreedy.select_worker
8 Dispatch	Bind, log decision, send TaskMessage	Coordinator._bind / _submit
9 Execution	Pinned worker process runs the stage (CPU/CUDA/remote)	core/worker_pool.py, apps/stages.py, apps/cuda_stages.py
10 Monitoring	Event log per task, per-core load, scheduling-time accounting	core/event_log.py, dashboard/data.py
11 Result collection	TaskResult -> DONE; RunSummary; output verified vs reference	Coordinator._handle_completion, apps.verify_output
12 Evaluation/feedback	Paired blocks, bootstrap CI, Holm; retraining for a new machine	analysis/*, scripts/study2_retrain.py

Key design rule learned from failures: order by predicted rank, place only on an idle worker (late, work-conserving binding). The earlier predictive EFT lost ~6% to a hand rule because of *early binding*, not bad predictions.

6. DAG, workload generation and dependencies

6.1 DAG implementation

core/dag.py: DAG stores tasks (id -> Task), supports topological_order(), dependents(tid), initial_ready_tasks(), is_complete(), and raises CycleError on cycles. Helper builders: build_independent_dag, build_chain_dag, build_fork_join_dag, build_mixed_dag.

6.2 The image-analysis application

apps/stages.py defines stages: **source, tile, denoise, edges, features, mosaic, aggregate, package**. The dashboard's default DAG has 68 tasks over 8 stages (dashboard/static/index.html); workload levels k = 1, 4, 16, 64 (dashboard/live.py LEVELS). CUDA variants exist for denoise, edges, features (apps/cuda_stages.py). img_source is CPU-only. In the dashboard demo, denoise and edges may also run on the emulated remote pool R (REMOTE in live.py).

6.3 Workload families used in experiments (study2/workload.py)

Builder	Shape	Why it exists
build_batch	64 identical chains source->denoise->edges->features + aggregate	Uniform control: tests 'do no harm'. No hidden cost, ~95% utilisation.
build_iters	Chains differing only in hidden denoise iterations	Cost hidden from size-keyed tables - where prediction can matter.
build_classes	Two-factor chains [count, side, iterations]; classes 48x(256px,2), 12x(256px,300), 12x(1024px,3)	Cost depends on size AND repeats; a size-only rule fails.
build_mixed	256/1024 px mix	Dropped.
Montage (workflow/adapter.py)	Real astronomy mosaic workflow	Stage-5 result: null (one dominant serial path, all tasks prefer P cores).
revival/workload.py	72 chains, 6 groups, two-level fan-in, 295 tasks	Contention / live-state tests.

7. ML runtime estimator and its metrics

7.1 Why predict runtime?

A scheduler cannot rank tasks or choose a worker without knowing how long each task will take on each worker. The estimator answers: *given this task and this pool, how many seconds?* **Feature** = an input number (image side in pixels, repeat count). **Target** = the measured kernel seconds. A model learns target from features.

7.2 Estimators in the repo (estimators/*, study2/param_model.py)

Name	Class	Idea
E0 constant	ConstantProfileEstimator	One number per workload.
E1 static table	StaticProfileEstimator	Median time keyed by (stage, size, pool).
E2 analytic	AnalyticSizeEstimator	Formula in size.
E3 history	MovingHistoryEstimator	Moving average of recent observed times.
E4 ridge	InterferenceRidgeEstimator	Ridge regression with occupancy/context features.
CatBoost / MLP	CatBoostEstimator, MlpEstimator	Tabular correction models (explored, not the final policy).
E5 oracle	ReplayOracleEstimator	Replays known true times - an upper bound, not deployable.
Param model (used by PRED_*)	ParamModelEstimator	Per (stage, pool) ridge regression on numeric task parameters + size + pairwise products.
Recalibrated	RecalibratedEstimator	Corrects a base estimator using observed residuals.

7.3 How ParamModelEstimator works

- **Training data:** a coverage session runs each stage on each pool with varied parameters; kernel seconds are read from event logs.
- **Features (decided from the data, not typed):** numeric entries of task.params plus size that vary in the logs; if two or more vary, their pairwise products too (cost often = size x repeats). seed/width/height/device/timeout are ignored.
- **Fit:** fit_group: standardise columns, ridge least squares (ridge=1e-9, solved by lstsq), return coefficients in raw units: seconds = a + sum(b_i x_i). **Revival form:** fit_group_log_power: log(time) = a + sum b_i log(x_i), fixes big relative error on the cheapest tasks.
- **Predict:** estimate(task, pool) looks up group key 'workload|pool' and takes a dot product (microseconds, no scheduling cost).
- **Unseen data:** unknown (workload, pool) raises KeyError - no silent fallback. New machine or pool -> one-command retrain (scripts/study2_retrain.py).

```
# study2/param_model.py (simplified)
coef = lstsq(standardised_features + ridge_rows, measured_seconds)   # fit_group
seconds = max(1e-6, coef . [1, x1, x2, x1*x2])                       # predict_group
```

Regularisation (ridge) adds a small penalty on coefficient size so the fit stays stable when features are correlated (size, repeats and their product are).

7.4 Metrics (formulas with a tiny example)

True times y = [10, 20, 30] ms; predicted p = [12, 18, 33] ms; errors e = [2, -2, 3].

Metric	Formula	Example	Meaning
MAE	mean(|p - y|)	(2+2+3)/3 = 2.33 ms	Average miss in ms. Robust, easy to explain.
RMSE	sqrt(mean((p - y)^2))	sqrt((4+4+9)/3) = 2.38 ms	Punishes big misses more.
R2	1 - sum(p-y)^2 / sum(y-mean(y))^2	1 - 17/200 = 0.915	Share of variation explained (1 = perfect).
Relative error	|p - y| / y	20%, 10%, 10%	Matters for cheap tasks.

Estimator accuracy is NOT scheduling performance. A model can be accurate and still give no makespan gain (no headroom), or be rough and still order tasks correctly. Verified hold-out: heavy classes within 8% (P 256px x300: predicted 0.605 s vs 0.565 s measured; 1024px x3: 0.122 vs 0.112); lightest class badly off (17.6 ms vs 4.1 ms on P). Power-law form cut denoise median error from 3.4 (P) / 1.0 (E) to 0.19 / 0.02 (revival measurement). Final-audit prediction accuracy numbers: **PENDING**.

8. Scheduling algorithms and differences

Two separate decisions: **task ranking** (which ready task first - order_ready) and **worker selection** (which worker - select_worker). Policies differ in how they do each.

Policy	Class (file)	Ranking	Worker selection	Type
B0_P_only	PoolOnlyScheduler (schedulers/simple.py)	FIFO	Idle P worker only; E and GPU never used	Static pool, late binding
B0_P_early	PoolOnlyEarlyScheduler	FIFO	P workers, queued on busy ones	Early binding control
B4 (B4_greedy_eligible)	GreedyEligibleScheduler	FIFO	First idle eligible worker, pool priority P, E (, R); no estimates	Dynamic, blind
B4_param_first / cost_first / largest_first	study2/policies.py (ParamFirstGreedy, CostFirstGreedy, LargestFirstGreedy)	By repeats / pixels x repeats / size, largest first	Same as B4	Hand-written rules that know the cost driver; non-predictive
EFT (EFT_E1_rc)	EarliestFinishTimeScheduler (schedulers/eft.py)	Ready order	Worker with earliest predicted finish incl. busy ones (early binding)	Dynamic, predictive
Rank-EFT	RankEftScheduler	Upward rank	EFT	Dynamic, predictive
HEFT	HeftScheduler (schedulers/heft.py)	Upward rank, whole DAG	Earliest finish time, full plan made before the run	**Static** plan
PEFT	PeftScheduler (schedulers/peft.py)	Optimistic cost table	Plan before the run	**Static** plan
HEFT_param_model, PEFT_param_model	built in study2/adapter.py	as HEFT / PEFT	Plans use ParamModelEstimator	Static, predictive
PRED_RANK_GREEDY	PredictedRankGreedy (study2/policies.py)	Predicted upward rank (longest remaining chain first)	Idle eligible worker with smallest predicted time	**Dynamic, predictive, work-conserving**
PRED_RANK_HOLD	same class, hold_min_s set	same	May hold a task for a much faster busy worker	Tested; no gain, 4x decision cost
Revival policy	revival/policy.py (ResourceStateAwareRankGreedy)	Rank	Adds live state (core load, residuals); falls back to B4 with logged reason	Gate failed - no headroom

_E2E variants (HEFT_E2E, PRED_RANK_GREEDY_E2E): estimators/e2e.py defines EndToEndTerms (adds file I/O / transfer terms to predictions). These belong to the ongoing audit; treat them as **AUDIT WIP / PENDING** until the audit finishes.

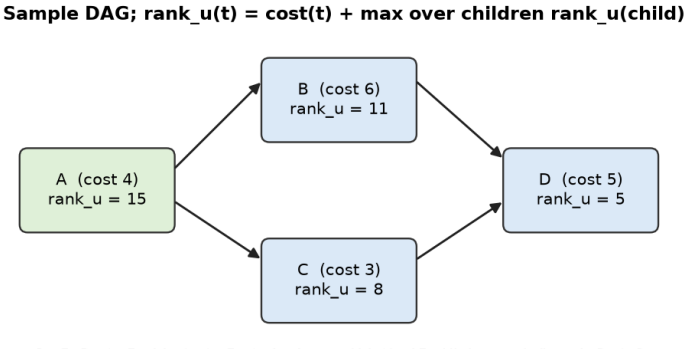
8.1 Differences in one line each

- **B4 vs PRED:** B4 is blind and uses FIFO + first idle; PRED orders by predicted remaining chain and picks the fastest idle worker.
- **HEFT vs PRED:** HEFT fixes the whole plan before running (static, early binding); PRED decides at runtime and never queues behind a busy worker.
- **PEFT:** like HEFT but looks one step ahead with an optimistic cost table; ~20% behind in Study 2; in the overnight check HEFT with the learned model was indistinguishable from PRED (-18.1% vs -19.0% vs B4) and PEFT much weaker (-2.6%).
- **HEFT result:** ~3% behind the hand rule in Study 2; static-plan overhead and early binding hurt.

8.2 Pseudocode of PRED_RANK_GREEDY (from the real code)

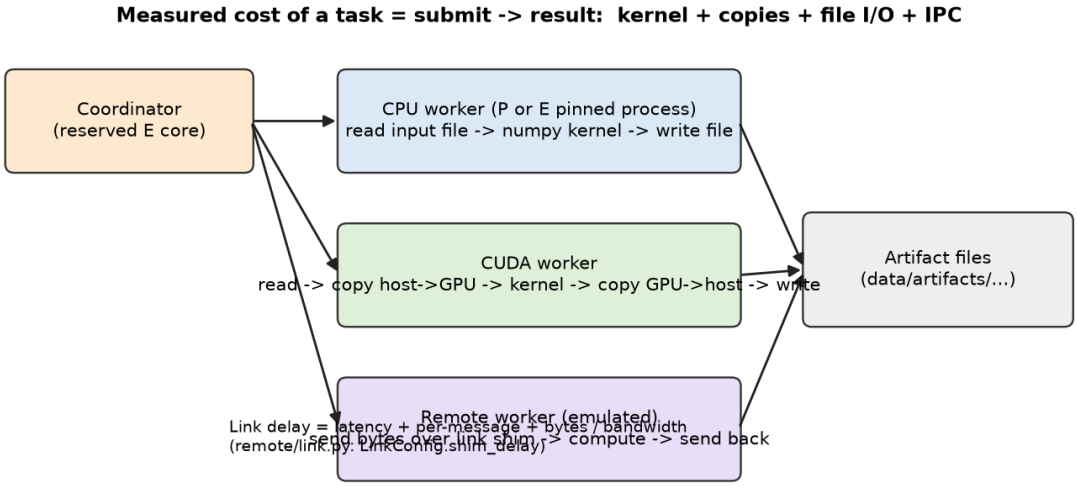
```
order_ready(ready):  sort ready tasks by upward rank (predicted times), largest first
select_worker(task):
    idle = eligible workers that are idle
    if none: return None                    # task waits (work-conserving)
    return argmin over idle of estimator.estimate(task, pool_of(worker))
# binds_only_idle = True: coordinator rejects binding to a busy worker
```

8.3 HEFT upward rank - numeric example



upward_rank (schedulers/heft.py) walks the DAG in reverse topological order: rank[t] = mean_cost(t) + max(edge(t,child) + rank[child]). With costs A4, B6, C3, D5 and zero communication: D=5, B=11, C=8, A=15. Higher rank = more work still behind it = schedule first.

9. CPU, GPU, CUDA, remote worker and offloading



9.1 Workers

- **P and E pools:** pinned processes (core/worker_pool.py), thread libraries pinned to 1 thread so tasks do not fight each other.
- **CUDA worker:** runs apps/cuda_stages.py via PyTorch; _submit sets device=cuda only when the chosen worker kind is cuda. A task is never forced onto a device it does not support: core/eligibility.py.
- **Remote worker (emulated):** remote/node.py (server), remote/proxy.py (client side), remote/protocol.py (framed messages with auth token), remote/link.py (LinkConfig.shim_delay = latency + per_message + bytes/bandwidth). All on one laptop; a Raspberry Pi is planned but **not tested**.

9.2 Why GPU can be faster or slower (VERIFIED, median ms incl. all costs, P / GPU)

Task	P-core	GPU	Reading
edges 256 px	6.2	7.2	Too small: copy + I/O overhead eats the gain.
edges 1024 px	45.0	14.8	GPU 3x faster (busy GPU: 18.8).
features 2048 px	52.4	22.2	Big data wins.
denoise x8, 1024 px	215	214	Tie; GPU busy -> 515 (P wins).
denoise x60, 1024 px	1562	1157	GPU wins quiet; busy 3570 -> P wins.

Breakdown of a GPU edges task at 512 px: kernel + host/device copies 33%, file I/O and codec 51%, IPC 8%. Host-to-device copy of the input is only 0.29 ms, but the GPU worker is occupied 9.1 ms - so **file I/O and messaging, not the kernel, dominate**. These costs are inside the measured wall time (submit to result) and were split out separately for analysis, so they must not be added twice.

Verdict (VERIFIED + ESTIMATE): single-task GPU speedups exist, but at workflow level 60% of the work is CPU-only (img_source), and in stored runs only 6.4% of dispatches had two or more idle workers (0.8% with both a GPU and CPU idle). Simulated oracle headroom was ~0.05%. So 'GPU offloading helps the whole workflow' is **not established**.

10. Project directory and important files

```
D:\cao project
  src/caosched/
    core/        coordinator.py  dag.py  task.py  ready_queue.py  eligibility.py
                 scheduler_base.py  worker_pool.py  event_log.py  runtime_config.py
    schedulers/  simple.py (B0,B4,...)  eft.py  rank_eft.py  heft.py  peft.py
    estimators/  base.py  profile.py  analytic.py  history.py  learned.py  tabular.py
                 oracle.py  recalibrated.py  e2e.py
    study2/      workload.py  param_model.py  policies.py  adapter.py   (main research code)
    revival/     policy.py  state.py  contention.py  oracle.py  gpuprofile.py ...
    apps/        stages.py  cuda_stages.py  image_pipeline.py  artifacts.py
    remote/      node.py  proxy.py  protocol.py  link.py
    workflow/    adapter.py  (Montage workflows)
    experiment/  runner.py  design.py  *_adapter.py  manifest.py
    analysis/    paired statistics, bootstrap CI, accuracy metrics
    dashboard/   server.py  live.py  data.py  static/(index.html app.js app.css)
    phase0/      CPU topology (P/E core discovery)
  scripts/       run_experiment.py  study2_*.py  revival_*.py  analyze_experiment.py ...
  tests/         ~500 tests
  config/  data/{results,events,profiles,models,traces}  docs/  manifests/
```

Open first	Why
PROJECT_BACKBONE.md	Whole story, all results, errors.
src/caosched/core/coordinator.py	The heart: dispatch loop.
src/caosched/study2/policies.py + param_model.py	The final policy and its model.
src/caosched/schedulers/simple.py	B0 and B4 baselines.
src/caosched/study2/workload.py	Experiment workloads.
docs/research_redesign/claims.json + STUDY2_REPORT.md	Claims with numbers.
docs/revival/FINAL_STATUS.md, GPU_CPU_OPPORTUNITY_REPORT.md	Revival + GPU findings.

11. Code walkthrough: workflow stage -> class/function

Workflow step	File	Class / function	Responsibility
Build DAG	study2/workload.py; apps/image_pipeline.py	build_batch / build_iters / build_classes; build_image_pipeline	Create Task nodes with dependencies, params, eligible pools
Eligibility	core/eligibility.py	resolve_eligibility, supported_kinds	Which pools can run each task; error if none
Run entry	core/coordinator.py	Coordinator.run	Main loop, health checks, timeouts, deadlock detection
Ready check	core/dag.py; coordinator	DAG.initial_ready_tasks; Coordinator._mark_ready	PENDING -> READY, push to ReadyQueue
Ordering	core/scheduler_base.py; study2/policies.py	SchedulerPolicy.order_ready; PredictedRankGreedy.order_ready	Rank ready tasks
Estimate	study2/param_model.py	ParamModelEstimator.estimate, predict_group	Predicted seconds per (task, pool)
Worker selection	schedulers/simple.py; study2/policies.py	GreedyEligibleScheduler.select_worker; PredictedRankGreedy.select_worker	Choose an idle eligible worker
Dispatch	core/coordinator.py	_dispatch_ready_tasks, _bind, _submit	Validate choice, DISPATCHED, send TaskMessage
Execution	core/worker_pool.py; apps/stages.py	_worker_loop; run_stage	Run the stage on pinned process / CUDA
Remote path	remote/proxy.py, node.py, link.py	proxy_loop; serve; LinkConfig.shim_delay	Emulated remote worker with link delay
Completion	core/coordinator.py	_handle_completion	DONE, log, unblock dependents, policy feedback
Monitoring/logging	core/event_log.py	EventLogger	JSONL events with timestamps
Evaluation	experiment/runner.py; analysis/*	run_session; paired stats	Blocks of conditions, bootstrap CI, Holm
Live demo	dashboard/live.py; server.py	execute; LiveManager; Handler.do_POST	Start a real run from the web UI

Key excerpt: the dispatch contract (core/coordinator.py)

```
ordered = self.scheduler.order_ready(list(ready), self.state)
for task_id in ordered:
    if late and not self.state.idle_workers(): return
    worker_id = self.scheduler.select_worker(task, self.state)
    if worker_id is None: continue                # task keeps waiting
    if late and not self.state.is_idle(worker_id):
        raise SchedulerContractError(...)          # late-binding policies may not queue
    self._bind(task_id, worker_id, ...)
```

Line by line: ask the policy for an order; stop if nobody is idle; ask for a worker; None means wait; the coordinator *enforces* the policy's promises (eligible pool, idle worker) and fails loudly otherwise; then bind and submit.

Key excerpt: B4 worker selection (schedulers/simple.py)

```
ok = set(eligible_workers(task, state, idle_only=True))
for pool in self._eligible_pools(task.workload):      # priority order, e.g. P, E, R
    idle = [w for w in state.idle_workers(pool) if w in ok]
    if idle: return idle[0]
return None
```

B4 never asks how long anything takes. B4 is a *comparison baseline* inspired by the eligible-worker dispatch idea of US patent 11,561,826 B1 (see docs/B4_PATENT_MAPPING.md); it is not an implementation of the patent.

AUDIT WIP (uncommitted, not final): the audit session has edits in core/coordinator.py (+15 lines), remote/node.py, experiment/remote_adapter.py, analysis/stats_paired.py, scripts/calibrate_link.py, study2/adapter.py and new remote tests. Treat remote-worker and statistics details as subject to change.

12. Frontend and complete demonstration

12.1 What the dashboard is

A local web app (src/caosched/dashboard/): a Python ThreadingHTTPServer (server.py, default port 8765, bound to 127.0.0.1) serving static/index.html, app.js, app.css. State-changing requests need a per-server token and matching Host/Origin headers.

Tab	What it shows
1 System overview	Live CPU load per logical processor; scheduler and application status; remote worker R0; Raspberry Pi and NVIDIA T4 panels (planned hardware, not real).
2 Application workflow	Replay an accepted run or press **Run live on this laptop**; the 68-task, 8-stage DAG, worker timeline, run summary, real application output image, selected-task details.
3 Scheduling comparison	Makespan by workload level; EFT_E1_rc vs B4 paired change; when transfer dominates (denoise); remote path vs local compute.
4 Hardware expansion	Intended architecture, why physical hardware is needed, implementation status.
5 Montage & GPU	Montage runs and the GPU spike data.

Dashboard policy choices are only two: B4 (B4_greedy_eligible_R) and EFT (EFT_E1_rc_R), at workload level k in {1, 4, 16, 64}. The dashboard does **not** expose PRED_RANK_GREEDY, HEFT or PEFT; those run through the experiment scripts.

12.2 What happens on 'Run live'

- Browser POSTs /api/live/start with {policy, k} and the token.
- LiveManager.start launches python -m caosched.dashboard.live as a subprocess (so core pinning never affects the server).
- live.execute: checks free RAM (>= 1.0 GiB), builds the pipeline, starts WorkerPool (P, E, remote slot), runs Coordinator.run, verifies the final artifact against a single-process reference, writes status.json.
- Browser polls /api/live and /api/run and animates the timeline. Output goes to data/dashboard_runs/, never into experiment records.

12.3 Demonstration script

```
1) Keep the charger plugged in; close heavy apps (need >= 1-2 GB free RAM).
2) cd "D:\cao project"
3) .venv\Scripts\python.exe -m caosched.dashboard        # default port 8765
4) Open http://127.0.0.1:8765   (do not use another host name)
5) Tab 1: show P/E core load = the heterogeneous machine.
6) Tab 2: pick replay of B4, press Play: explain DAG, timeline, which pool each task used.
7) Run live: policy B4, k=4 -> then EFT, k=4. Compare makespan and tasks per pool.
8) Tab 3: show the stored paired comparison (EFT vs B4) and say what it means.
9) Tab 4/5: say clearly: Raspberry Pi / T4 / real remote are NOT tested.
```

Step 3 is verified against dashboard/__main__.py: python -m caosched.dashboard [--port 8765] [--open]. The team's own 3-5 minute talk script is FACULTY_DEMO_SCRIPT.md in the project root (untracked file).

13. Experiments, metrics and verified results

13.1 Setup

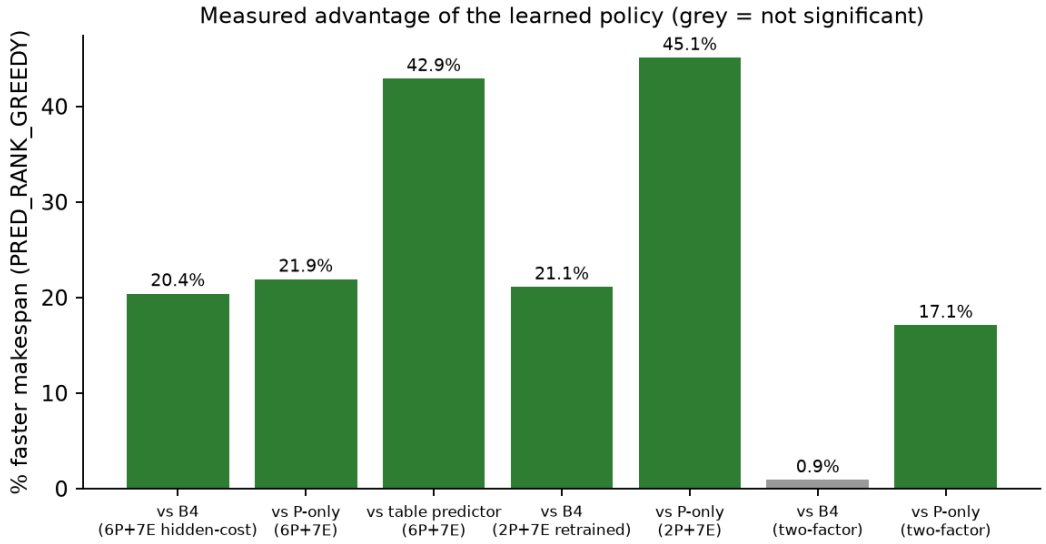
- Hardware (PROJECT_BACKBONE.md): Windows 11 laptop, 6 P cores + 7 E cores + 1 coordinator core, 15.7 GB RAM, RTX 3050 6 GB laptop GPU, torch 2.11+cu128. Remote = loopback emulation.
- Method: conditions run in random-order **blocks**; one worker pool reused for all conditions (an early pilot was confounded by pool rebuilds); pre-registered protocol written before each run; paired log-ratio bootstrap CI; Holm correction; guards: scheduling cost < 25% of makespan, AC power, >= 0.8 GB RAM.
- Training vs held-out: model fit on coverage logs, policy chosen on pilot logs, reported on confirmatory logs.

13.2 Metrics (what is recorded)

Metric	Definition	Recorded?
Makespan	last finish - run start	Yes (RunSummary)
Scheduling time	time in order_ready + select_worker	Yes (scheduling_time_seconds); ~1% of makespan
Init / setup time	init_s and setup_s measured in Coordinator.run	Yes
Queue wait	t_dispatched - t_ready per task	Yes (queue_wait_seconds)
% improvement	(T_base - T_new) / T_base x 100	Computed in analysis; positive = new faster
Paired CI / p-value	Bootstrap on per-block log ratios; Holm across claims	Yes (analysis/stats_paired.py)
MAE / RMSE / R2	Section 7.4	Prediction accuracy analysis
Utilisation	busy time / makespan per worker	Derivable from event logs

Fair static vs dynamic comparison: static planners (HEFT, PEFT) spend time planning before the run; dynamic ones decide during it. A fair comparison counts both inside the same end-to-end timing boundary and gives both the same eligible resources. This end-to-end boundary work is part of the ongoing audit: **PENDING**.

13.3 Verified results (from PROJECT_BACKBONE.md; raw logs under data/)



Experiment	Result	Status
Montage (stage 5)	Prediction-aware EFT 1.6% slower than static P-only (285 s); HEFT +1.5%, PEFT +1.7%	VERIFIED null
Hidden-cost, 6P+7E (30 blocks)	PRED 20.4% faster than B4, 21.9% than P-only, 42.9% than table predictor; 1.6% behind hand rule that knows the parameter	VERIFIED
Hidden-cost, 2P+7E retrained (20 blocks)	21.1% faster than B4, 45.1% than P-only, 36.6% than table predictor; level with hand rule (0.4%)	VERIFIED

Experiment	Result	Status
Uniform batch no-harm (30 blocks)	Level with B4 (+0.45%), 26.1% faster than P-only	VERIFIED
Two-factor (confirmatory 41 blocks)	Size-only rule 17.4% slower, P-only 17.1% slower (supported). B4 0.9% slower, CI -0.9% to +2.8% (not significant). Equivalence to hand formulas not resolved	VERIFIED
Revival oracle gate (13 blocks)	Headroom of best simple rule over oracle -1.24%; required >= 5% -> FAILED. Live-state idea not supported	VERIFIED negative
Revival no-harm (31 blocks, CUDA worker)	v1 +2.2-2.6% slower than B4 (cause: snapshot per completion); v2 +0.7-1.0% after fix	VERIFIED
GPU/CPU profiling (Phase 1)	Opportunity not established (G1 fails at 512 px, G2 fails, G3 only unattainable bound)	VERIFIED + ESTIMATE
Overnight check 2026-10-27 (quoted from FACULTY_DEMO_SCRIPT.md; raw files listed in RESULTS_TABLE.csv; not re-checked here)	20 never-used chain orders: learned policy 19.3% / 20.5% faster than B4, faster in all 20 (exploratory). Dose-response: ~1-3% gain when heavy chains cost 7-20x a light one, ~20% at 60x. HEFT with the same learned model -18.1% vs ours -19.0% (indistinguishable); PEFT only -2.6%. Scheduling cost 3-5% of the run. Single GPU stages up to ~15x faster at large tiles, no workflow-level GPU gain shown.	VERIFIED by project records (secondary)
Hardware: Raspberry Pi / real remote	Not tested	NOT EVALUATED
Final audit experiments (CPU-only vs CPU+GPU, static vs dynamic, E2E overhead, significance)	Calibration run in data/results/final_20261101 failed to start (design file not found) and was being retried; no complete final result exists yet	**PENDING**

Do not say: 'our scheduler is 20% better than everything'. The 19-22% gains over B4 came from workloads where hidden task parameters make ordering decisive (CPU-only P/E pools), not from CPU/GPU heterogeneity. Test status in the audit run: 515 passed, 1 failed (a RAM-guard test that refuses to run with <2 GB free RAM), 1 skipped - **AUDIT WIP**, final count pending.

14. Base papers and related work

The code and docs name these sources. I did not do new literature research; check the project's own docs/revival/RELATED_WORK.md and PAPER_OUTLINE.md before quoting details.

Source	What it is / what we take	Difference in our work
Topcuoglu, Hariri, Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing. IEEE TPDS 13(3), 2002	HEFT: upward rank + earliest finish time. We implement it (schedulers/heft.py) as a static baseline and reuse upward rank for PRED.	We rank dynamically from *learned* predictions and bind late to idle workers.
Arabnejad, Barbosa. List scheduling algorithm for heterogeneous systems by an optimistic cost table. IEEE TPDS 25(3), 2014	PEFT: optimistic cost table (schedulers/peft.py).	Static plan; ~20% behind in Study 2, but only 2.6% gain over B4 in the overnight check (different experiments).
US patent 11,561,826 B1 (Xilinx)	Eligible-worker heterogeneous dispatch concept; basis for the B4 baseline (docs/B4_PATENT_MAPPING.md).	B4 is a baseline only, not the patent's implementation.
Lotaru (arXiv 2309.06918, FGCS 2024); Lu and Reda (listed in RELATED_WORK.md); Ilyushkin and Epema CCGrid 2018	Runtime prediction / workflow scheduling related work recorded by the project.	No novelty claimed over these (docs/revival/RELATED_WORK.md).

Aspect	Base HEFT/PEFT	Our implementation
Timing of decisions	Static, before run	Dynamic (PRED) plus static baselines
Runtime knowledge	Assumed cost matrix	Learned per-(stage, pool) ridge model
Resources	Abstract processors	Real P/E pools, CUDA worker, emulated remote
Evaluation	Simulation, schedule length	Real runs, paired blocks, bootstrap CI, Holm
Performance evidence	Superior in their simulations	Beats naive/static; ties best simple rules; GPU/live-state not shown

NVIDIA already provides CUDA streams, graphs and libraries for work *inside* a GPU program. CAO operates one level higher: placing whole workflow tasks across heterogeneous workers. See NVIDIA_VS_CAO_AUDIT.md in the project root for the team's detailed comparison (not re-verified here).

15. Limitations, future work and faculty questions

15.1 Limitations (be upfront)

- Gains over blind greedy B4 appear only when cost is hidden in task parameters; none on uniform, two-factor or Montage workloads.
- Never beat the best hand-written rule that knows the cost driver (level or slightly behind).
- GPU: single-task speedups do not give workflow speedups; few real dispatch choices; NVML not installed so no per-decision GPU telemetry.
- Remote worker is emulated on one laptop; Raspberry Pi / NVIDIA T4 hardware not tested.
- Estimator poor on the cheapest tasks; unseen (stage, pool) is an error; retraining needed per machine.
- Limited by free RAM (WSL VM and background services); few repetitions in the GPU profile (8).
- Final audit (E2E timing, fair static baselines, CPU+GPU, significance) is **PENDING**.

15.2 Future work

- New workload with real choices (fewer ready tasks than workers, GPU-dominated work), screened by simulation first.
- Real Raspberry Pi / remote validation (one-command retraining exists).
- Better light-task prediction (power-law form exists).
- Finish audit and update the results chapter.

15.3 Likely faculty questions and answers

Q1. What problem do you solve?

Choosing order and worker for dependent tasks on a machine with unequal workers, to minimise total finish time (makespan).

Q2. Why a DAG?

Real workflows have dependencies; a DAG captures them and gives critical path and ready-task logic.

Q3. Why ML?

Fixed tables cannot capture hidden parameters like repeat counts. A small regression predicts per-pool time cheaply (microseconds).

Q4. Which ML model?

Per-(stage,pool) ridge regression on task parameters and size, plus pairwise products; a log-power variant for cheap tasks. Not a neural network.

Q5. How is PRED different from B4?

B4: FIFO and first idle worker, no estimates. PRED: orders by predicted remaining chain, picks the idle worker with smallest predicted time.

Q6. Why not only HEFT?

HEFT plans statically and binds early; ~3% behind the hand rule in Study 2. In the overnight check HEFT with our learned model matched PRED (-18.1% vs -19.0%), so the real contribution is the learned model, not only the policy. Plans cannot correct errors at runtime.

Q7. How is worker selection done?

Among idle eligible workers, the minimum of estimator.estimate(task, pool); eligibility comes from the task's allowed pools.

Q8. What is late binding and why does it matter?

Binding a task only to an idle worker; keeps workers busy and avoids queueing behind slow workers (early binding cost ~1.7% on Montage).

Q9. Why use a GPU if transfer is costly?

Large tasks gain (edges 1024 px: 45 ms to 14.8 ms). Small ones lose. The scheduler/estimator must include copy and I/O cost.

Q10. What happens when the GPU is busy?

Measured: GPU denoise gets 2.2-3.1x slower and the best device flips to P. The live-state policy that reacts to this showed no headroom.

Q11. What does NVIDIA already provide?

Streams, graphs and libraries inside one GPU program. We schedule whole workflow tasks across CPU pools, GPU and remote workers.

Q12. How do you protect against a bad prediction?

Late binding bounds the damage (no queueing); revival policy falls back to B4 with logged reasons; scheduling cost guard 25%; unseen groups raise an error.

Q13. How do you compare static and dynamic fairly?

Same workload, same eligible resources, ONE ridge cost model on both sides, and the end-to-end makespan from the start of run() to the last completion, so the planning time of a static plan is counted. Done in the final benchmark (Section 16): PRED is 5.2% faster than static HEFT on the CPU and 12.2% with the strict GPU set.

Q14. Why end-to-end makespan?

Users care about total time including setup and scheduling, not just kernel time.

Q15. Why bootstrap CI and Holm?

Runs are noisy; paired blocks remove drift; bootstrap gives intervals; Holm controls false positives across many claims.

Q16. What is genuinely new?

Be modest: no broad novelty claimed. Defensible: a generic, retrainable parameter-aware runtime model with rank + late-binding policy, evaluated with pre-registered negative and positive results on a real heterogeneous runtime.

Q17. What does the project prove?

Prediction-aware scheduling beats static and naive policies by 17-45% where costs vary; it identifies when it does not help.

Q18. What does it not prove?

Superiority over B4 in general, GPU workflow speedup, live-state benefit, real remote or Raspberry Pi benefit.

Q19. Limits of remote emulation?

One machine, loopback; delay is a formula (latency + per-message + bytes/bandwidth). Real networks have jitter, loss and different CPUs.

Q20. What is makespan vs speedup?

Makespan = total time. Speedup = T_base / T_new. 20% faster means T_new = 0.8 T_base.

Q21. Why did Montage fail?

One dominant serial path and every task prefers P cores, so placement choices barely matter.

Q22. Is estimator accuracy the same as scheduling gain?

No. Accurate predictions help only if there is headroom for a different decision.

Q23. What is still pending?

Nothing in the final audit: it is complete (Section 16). Open by design: Raspberry Pi and real-network results, a better GPU cost model, a second workload family, the GitHub push and the IPR decision.

15.4 Presentation order (10 minutes)

Problem and analogy -> heterogeneous machine -> DAG and architecture -> workflow loop -> B4 vs PRED -> ML model -> live dashboard demo -> verified results (including the honest nulls) -> limitations -> conclusion: 'predictions help when cost is hidden; we know where it does not.'

Updated 3 November 2026: the final audit is finished; Section 16 holds the final static-versus-dynamic results, the CPU/GPU conclusion, the new policies, the dashboards v2 and v3 and the test count. Earlier sections are unchanged and describe the earlier study; where they differ, Section 16 takes precedence.

16. Final static-versus-dynamic benchmark and dashboards v2 and v3 (update, 3 November 2026)

Read this first. Sections 1 to 15 were written before the final benchmark. This section is the current state and takes precedence. All numbers below are read from the stored analysis files of data/results/final_20261101 (generated into CAO_FINAL_EXPERIMENT_REPORT.md).

16.1 The question and the answer in plain words

Question: does a dynamic scheduler (decides while the workflow runs) finish sooner than a static plan (decided before the run) when both use the same cost model, once planning and start-up time are counted? Answer on the primary workload (16 heavy chains of 64, 24 blocks): **yes**. The dynamic policy PRED_RANK_GREEDY is 17.8% faster than blind B4 on the CPU, 21.2% with the strict GPU set, and 5.2% (CPU) and 12.2% (strict GPU) faster than the static HEFT plan. A simple rule that sorts the queue by the cost parameter (B4_param_first) is within 1% of PRED, so most of the gain comes from using the task parameter, not from the learned rank.

16.2 Policies added or changed

Policy	Kind	What it does
B4	dynamic	first idle eligible worker, no estimate (the base idea)
B4_param_first	dynamic	B4 with the queue sorted by the cost parameter (hand rule)
PRED_RANK_GREEDY	dynamic	predicted upward rank order; idle worker with least predicted kernel time
PRED_RANK_GREEDY_E2E	dynamic	as PRED, minimising predicted occupancy: transfer + kernel + set-up + inter-process time
HEFT_param_model, PEFT_param_model	static	plan before the run from the same model
HEFT_E2E	static	HEFT with the end-to-end terms in the costs
HEFT_oracle	static, reference	plan from class-median costs; reference only, not a bound

Where in code: src/caosched/final/policies.py (policies), final/gpu_region.py (GPU eligibility from verified spike rows), final/adapter.py (run adapter), scripts/final_*.py (calibration, evaluation, analysis, report), tests/test_final_*.py. Timing rule: docs/final/AMENDMENT_F1_timing.md (end-to-end makespan = start of run() to last completion; init and policy set-up logged separately).

16.3 CPU only versus CPU + GPU (the three configurations)

cpu: P and E cores only. **gpu** (strict): the CUDA worker may run only stages whose speed-up was verified on the original spike record (edges 7.68x, features 2.88x, denoise at 8 iterations 1.16x; denoise at 1 iteration dropped). **ext**: strict plus denoise at the iteration counts verified on the extended record (6, 15, 40, 120 iterations; 2 dropped). None is GPU-only. Static versus dynamic is independent of the hardware set: it is about when the decision is made.

Policy	CPU only	Strict GPU	Extended GPU
B4 (blind, dynamic)	4.010	4.075	4.123
B4 param-first (rule, dynamic)	3.310	3.218	3.069
PRED (dynamic)	3.297	3.213	3.050
PRED-E2E (dynamic)	3.318	3.212	3.033
HEFT (static)	3.478	3.660	3.114
HEFT-E2E (static)	3.499	3.512	3.173
PEFT (static)	4.093	4.050	3.689

Makespan in seconds, geometric mean over 24 blocks. Effect of the GPU for PRED: strict set 2.6% faster than CPU only, extended set 7.5%. For blind B4 the GPU does not help (+1.6%, inconclusive). The GPU cost model has 23% to 31% median error (CPU stages 4% to 11%), so GPU placement is the weakest part.

16.4 The ten pre-stated contrasts (Holm-adjusted)

Id	Contrast	Effect % [95% CI]	Holm p	Result
C1	PRED vs B4 (CPU)	–17.8 [–18.8, –16.7]	0.001	faster
C2	PRED-E2E vs PRED (CPU)	+0.6 [+0.1, +1.3]	0.141	equivalent (within ±2%)
C3	PRED vs HEFT (CPU)	–5.2 [–5.9, –4.6]	0.001	faster
C4	PRED-E2E vs HEFT-E2E (CPU)	–5.2 [–5.9, –4.5]	0.001	faster
C5	PRED vs HEFT (strict GPU)	–12.2 [–13.4, –11.1]	0.001	faster
C6	PRED-E2E vs HEFT-E2E (strict GPU)	–8.5 [–9.7, –7.6]	0.001	faster
C7	PRED-E2E vs PRED (strict GPU)	0.0 [–0.7, +0.5]	0.960	equivalent (within ±2%)
C8	PRED vs B4 (strict GPU)	–21.2 [–22.6, –19.7]	0.001	faster
C9	PRED, strict GPU vs CPU	–2.6 [–3.3, –1.7]	0.001	faster
C10	B4, strict GPU vs CPU	+1.6 [+0.2, +3.0]	0.141	inconclusive

16.5 What the results do not show

- A hand rule that knows the cost parameter is within 1% of PRED in every configuration.
- With no spread in task cost (uniform workload) all policies tie; with the smallest cost ratio and the strict GPU set PRED is 1.1% slower than B4.
- End-to-end cost terms change nothing (C2, C7 equivalent within 2%); a hybrid B4/PRED selector would gain at most 1.1% (maximum over 17 cells), so none was built.
- An earlier session found HEFT level with PRED (+1.10%); the final session finds PRED about 5% faster; the difference was not isolated and both are reported.
- One machine, one workload family, one CUDA worker; the remote worker was not rerun with the final policies; no Raspberry Pi result exists.

16.6 Dashboards (new files; the original dashboard is unchanged)

Version	Command (project root)	What is new
original	.\.venv\Scripts\python.exe -m caosched.dashboard --port 8765 --open	unchanged; CPU load bars stay 0 after the first request (old psutil baseline bug)
v2	.\.venv\Scripts\python.exe -m caosched.dashboard_v2 --port 8766 --events-dir data\results\final_20261101\eval\sessions --open	GPU CONNECTED with live load/memory/temperature/power, Pi card NOT CONNECTED, final results with hover tooltips on screen 3, CPU load fixed
v3	.\.venv\Scripts\python.exe -m caosched.dashboard_v3 --port 8770 --events-dir data\results\final_20261101\eval\sessions --open	v2 plus screen 6: run one live block of the final benchmark with any policy on CPU or CPU+GPU (1 to 2 minutes, needs 2.4 GB free RAM)

A live run on screen 6 is a demonstration of one block, not an experiment result: its makespan (about 4 s) is not comparable with the benchmark (about 3 s); the cause was not isolated (it includes pool and CUDA start-up).

16.7 Tests and reproducibility

521 tests: 520 pass; one (tests/test_profile_montage.py::test_profile_format_csv_fit_and_cleanup) refuses to start when free RAM is below its own 2 GB gate and failed at 1.44 GB free; the test passed in the earlier full run. Pipeline: scripts/final_stage1.ps1, final_stage2.ps1, final_analyze.py, final_results_table.py, final_cross_workload.py, final_hybrid_check.py, final_build_report.py. Submission package: SUBMISSION_FINAL.